

Introduction to Probabilistic Machine Learning

Information Theory

Ralf Herbrich

Overview

1. Basics of Information Theory
2. Probability and Compression: Arithmetic Coding
3. Distance Measures for Probabilities Revisited

**Introduction to
Probabilistic Machine
Learning**

Unit 12 – Information Theory

1. **Basics of Information Theory**
2. Probability and Compression: Arithmetic Coding
3. Distance Measures for Probabilities Revisited

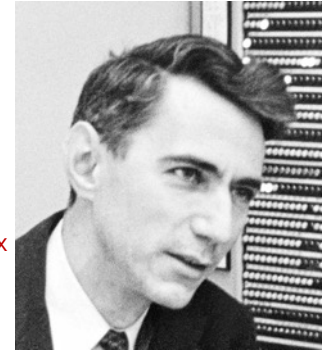
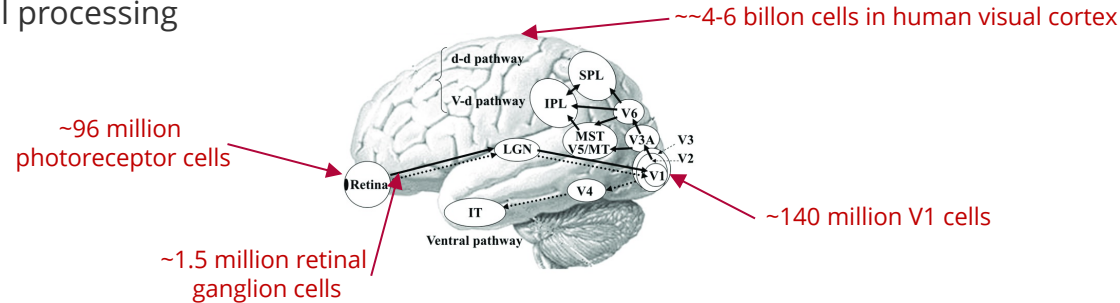
**Introduction to
Probabilistic Machine
Learning**

Unit 12 – Information Theory

What is Information?

- **Information:** Facts, data or knowledge communicated or received about a particular situation, person, event or topic

- Provides understanding or allows for informed decisions
- Encoding and decoding information for transmission and interpretation is key!
- **Example:** Visual processing



Claude Shannon
(1913 – 2001)

Introduction to Probabilistic Machine Learning

Unit 12 – Information Theory

- **Information Theory:** Mathematical study of the quantification, storage, and communication of information.

- **Information:** A set of messages where the goal is to send the messages over a noisy channel, and to have the receiver reconstruct the message with low error probability
- Less than 100 years old and established by Claude Shannon in 1948

Motivating Example: Information and Coin Tosses

■ Scenario 1:

- A coin toss with uncertain outcome modelled via $X \sim \text{Ber}(p)$
- $h(x|p)$ is the information received by observing the value of x if you know p
- **Question:**
 - How much is $h(1|100\%)$ when the success probability was 100%?
 - What's the relation between $h(1|99\%)$ and $h(1|1\%)$?
- **Conclusion:** $h(x)$ is monotonically decreasing in $P(x)$

■ Scenario 2:

- Two independent coins are tossed modelled via $P(x, y) = P(x) \cdot P(y)$
- **Question:** In what relation does $h(x, y)$ stand to $h(x)$ and $h(y)$?
- **Conclusion:** If $P(x, y) = P(x) \cdot P(y)$ then $h(x, y) = h(x) + h(y)$

$$h(x, y) = h(x) + h(y)$$

$$h(x, y) > h(x) + h(y)$$

$$h(x, y) < h(x) + h(y)$$

Introduction to
Probabilistic Machine
Learning

Unit 12 – Information Theory

$$h(x) = -\log_b(P(x))$$

Measure of Information: Entropy

- **Observation:** Information $h(x)$ is only defined for a particular outcome x !
- **Entropy.** The entropy of a random variable X is the average level of information *inherent to the variable's outcomes and is defined by* ($b > 1$)

Example

X	red	green	blue	yellow
$P(X = x)$	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{8}$	$\frac{1}{8}$

$$H_b[X] := - \sum_x P(X = x) \cdot \log_b(P(X = x)) = E_{X \sim P}[-\log_b(P(X))]$$

$\nwarrow$
 $-h(x)$

$$H_2[X] = \frac{1}{2} + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 3 = 1.75$$

Interpretations of Entropy:

1. Minimum number of random bits to draw outcomes from $X \sim P$ on average
 $P(X = \text{red}) \cdot 1 + P(X = \text{green}) \cdot 2 + P(X = \text{blue}) \cdot 3 + P(X = \text{yellow}) \cdot 3 = H_2[X]$
2. Average number of bits to store a compressed outcome of $X \sim P$

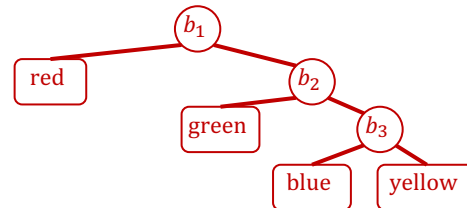
$$\text{average number of bits} = \sum_{x \in \{\text{red}, \text{blue}, \text{green}, \text{yellow}\}} P(X = x) \cdot |C(x)| = H_2[X]$$

3. Minimum number of yes/no questions on average to guess a draw from $X \sim P$

1. Is x red?
2. Is x green?
3. Is x blue?



$$P(X = \text{red}) \cdot 1 + P(X = \text{green}) \cdot 2 + P(X = \text{blue}) \cdot 3 + P(X = \text{yellow}) \cdot 3 = H_2[X]$$



X	$C(X)$
red	0_2
green	10_2
blue	110_2
yellow	111_2

Entropy: An Axiomatic Introduction

- **Khinchin (1957).** *Entropy* $H_b[X]$ as a measure of information of a random variable X follows from the following four axioms:

1. $H_b[X]$ depends only on the probability distribution P of X .
2. $H_b[X]$ is maximal for the uniform distribution P of X .
3. $H_b[Y] = H_b[X]$ if X and Y have the same non-zero probabilities.
4. For any random variables X and Y ,

$$H_b[X, Y] = H_b[X] + \sum_x P(X = x) \cdot H_b[Y | X = x] \quad \leftarrow H_b[Y|X]$$

- The last axiom leads to the logarithm because it says that for independent distributions $P(X = x, Y = y) = P(X = x) \cdot P(Y = y)$

$$H_b[Y|X] = H_b[Y] \Leftrightarrow H_b[X, Y] = H_b[X] + H_b[Y]$$



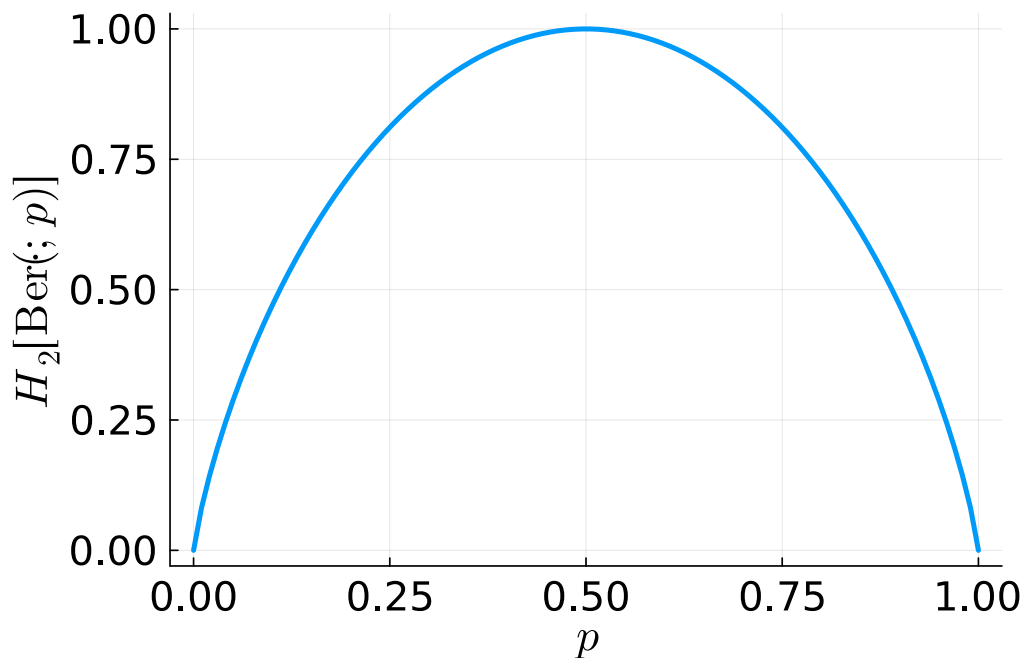
Aleksandr Khinchin
(1894 – 1959)

**Introduction to
Probabilistic Machine
Learning**

Unit 12 – Information Theory

Example: Binary Entropy

$$H_2[\text{Ber}(\cdot; p)] = -(p \cdot \log_2(p) + (1 - p) \cdot \log_2(1 - p))$$



Overview

1. Basics of Information Theory
- 2. Probability and Compression: Arithmetic Coding**
3. Distance Measures for Probabilities Revisited

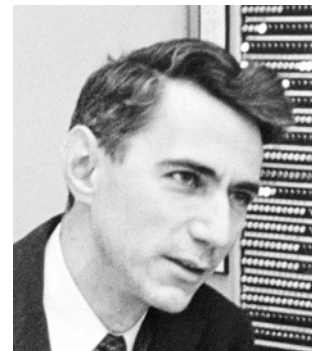
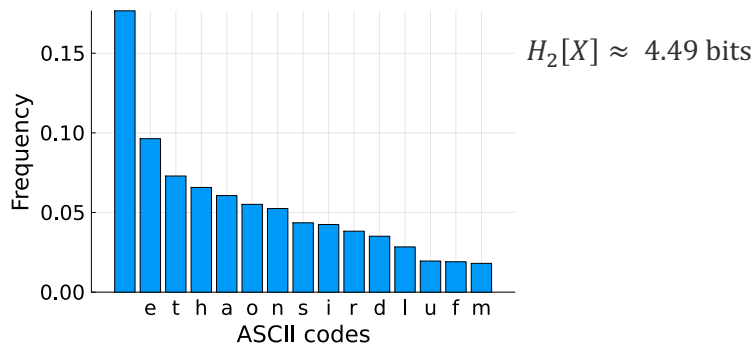
**Introduction to
Probabilistic Machine
Learning**

Unit 12 – Information Theory

Entropy and the Noiseless Coding Theorem

- **(Shannon 1948).** *N independent and identically distributed random variables each with binary entropy $H_2[X]$ can be compressed into more than $N \cdot H_2[X]$ bits with negligible risk of information loss, as $N \rightarrow \infty$; but if they are compressed into fewer than $N \cdot H_2[X]$ bits it is virtually certain that information will be lost.*
- **Application** in data compression when modelling the value X of a byte modelled as a random variable over $n = 256$ values
 - **Random bytes:** If we assume each byte value is equally likely then we require

$$H_2[X] = -\sum_{i=1}^{256} \frac{1}{256} \log_2 \left(\frac{1}{256} \right) = -\log_2 \left(\frac{1}{256} \right) = 8 \text{ bits}$$
 - **Random letters from the English alphabet:**



Claude Shannon
(1913 – 2001)

**Introduction to
Probabilistic Machine
Learning**

Unit 12 – Information Theory

Noiseless Coding Theorem: Example

- Scenario:** $X \in \{A, B, C, D, E, F, G, H\}$ with probabilities $\left\{\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{64}, \frac{1}{64}, \frac{1}{64}, \frac{1}{64}\right\}$

$$H_2[X] = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{16} \cdot 4 + 4 \cdot \frac{1}{64} \cdot 6 = 2$$

Fixed-Length Code

Each binary string of length 3

x	Code	$P(x)$	Length	$E[\text{Length}]$
A	000	1/2	3	96/64
B	001	1/4	3	48/64
C	010	1/8	3	24/64
D	011	1/16	3	12/64
E	100	1/64	3	3/64
F	101	1/64	3	3/64
G	110	1/64	3	3/64
H	111	1/64	3	3/64

Decode: 1 1 1 0 0 1 0 1 0
 H B C

$$\frac{192}{64} = 3 \text{ bits}$$

Prefix Code

Unique binary prefix of consecutive 1's for each unique probability

x	Code	$P(x)$	Length	$E[\text{Length}]$
A	0	1/2	1	16/32
B	10	1/4	2	16/32
C	110	1/8	3	12/32
D	1110	1/16	4	8/32
E	111100	1/64	6	3/32
F	111101	1/64	6	3/32
G	111110	1/64	6	3/32
H	111111	1/64	6	3/32

Decode: 1 1 1 0 0 1 0 1 0
 D A B B

$$\frac{64}{32} = 2 \text{ bits}$$

Optimal Codes: Arithmetic Coding

- Typically, a code is a 1-to-1 mapping from symbols x to b -ary strings $C(x)$
- The expected length (in bits) of transmitting a symbol is given by

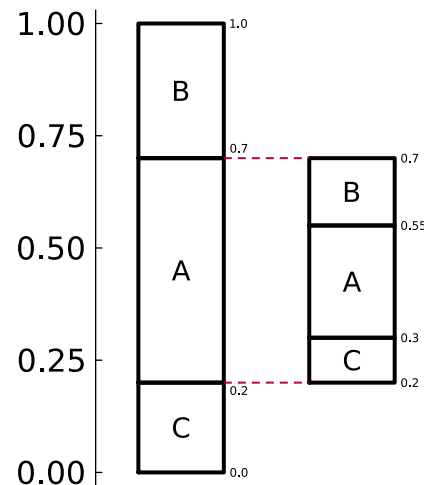
$$E[C(X)] = \sum_x P(X = x) \cdot |C(x)|$$

$= H_b[X]$ if $|C(x)| = -\log_b(P(X = x))$

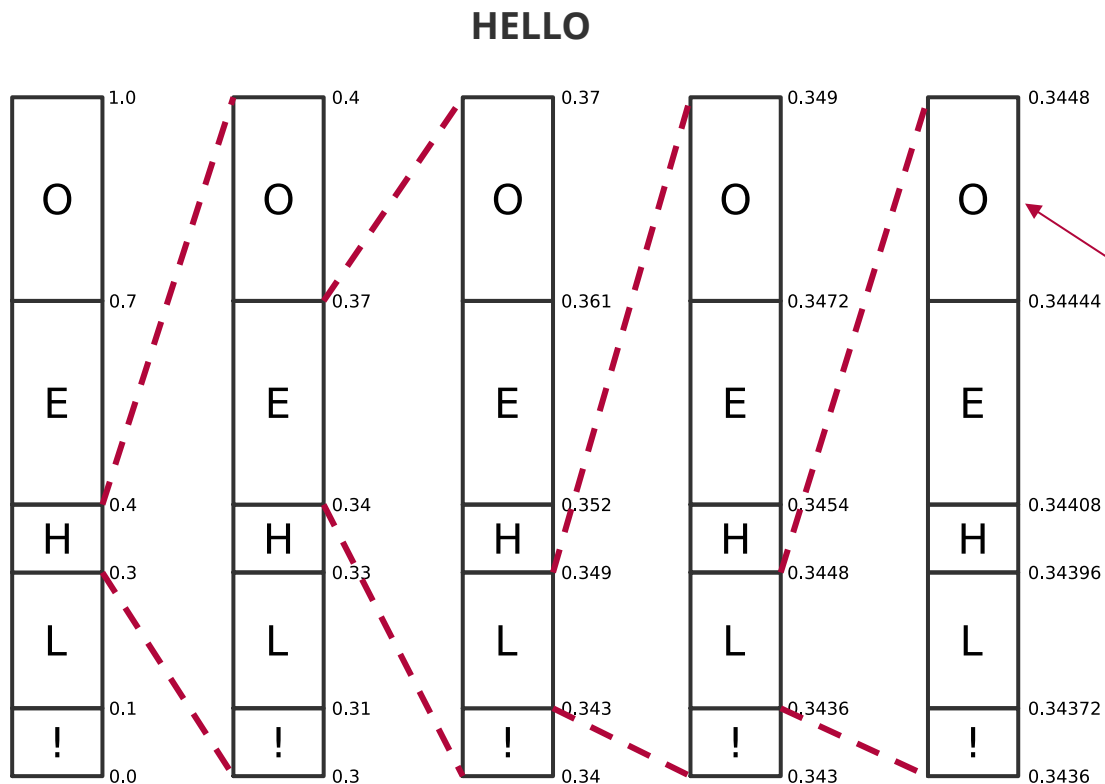
- Thus, the optimal length of the code for x would be $-\log_b(P(X = x))$
 - In general, this will be impossible because $-\log_b(P(X = x))$ is not an integer for all x !
- **Arithmetic Coding:** Main Steps
 1. **Modelling:** We have a separate probabilistic model P for a sequences of symbols which maps each sequence $x_1, \dots, x_n$ to a unique interval $I(x_1, \dots, x_n) \subseteq [0,1)$
 2. **Encoding:** Encode the unique description of the interval $I(x_1, \dots, x_n)$
 3. **Decoding:** Decode to the sequence $x_1, \dots, x_n$ that matches the description of the stored interval

The length of $I(x_1, \dots, x_n)$ equals $P(x_1, \dots, x_n)$!

Fixed-Model $\prod_i P(X = x_i)$



Arithmetic Coding: Fixed Probability Model



x	$P(x)$	Range(x)
!	0.1	[0,0.1)
L	0.2	[0.1,0.3)
H	0.1	[0.3,0.4)
E	0.3	[0.4,0.7)
O	0.3	[0.7,1.0)

$\text{Range}(\text{"HELLO"}) = [0.3444, 0.3448]$

$C(\text{"HELLO"}) = 3446$

$\log_{10} P(\text{"HELLO"}) \approx 3.44$

**Introduction to
Probabilistic Machine
Learning**

Unit 12 – Information Theory

Arithmetic Coding: Practical Considerations

1. Binary base: Instead of base-10, in practice we use binary interval arithmetic

- A binary string $\mathbf{b} \in \{0,1\}^n$ encodes the number in $[0,1)$ via

$$N(\mathbf{b}) = b_1 \cdot \frac{1}{2} + b_2 \cdot \frac{1}{4} + \dots + b_n \cdot \frac{1}{2^n} = \sum_{i=1}^n b_i \cdot 2^{-i}$$

- A binary string $\mathbf{b} \in \{0,1\}^n$ of length n encodes all values of probabilities $\left[N(\mathbf{b}), N(\mathbf{b}) + \frac{1}{2^n}\right)$

2. Fixed-Point Arithmetic. Arithmetic coding requires arbitrary precision multiplication and addition in binary numbers on finite precision CPUs

- If the bit-representation of the left and right range have common leading bits, they can be "shifted off" because successive probabilities can only get smaller!

3. Adaptive Probability Models. Adaptive probability model P can lead to larger ranges and higher compression!

$$P(x_1, \dots, x_n) = P(x_1) \cdot P(x_2|x_1) \cdot \dots \cdot P(x_n|x_1, \dots, x_{n-1})$$

Arithmetic Coding: Algorithm

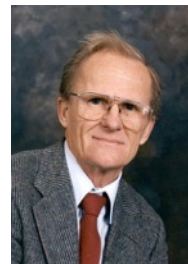
■ Given:

- Data as a sequence of tokens with K values: $(x_1, x_2, \dots, x_m) \in \{1, \dots, K\}^m$
- A probability model for each token: $P(x_j | x_1, x_2, \dots, x_{j-1}) \in [0, 1]$

■ Idea: Map a data stream $\mathbf{x} = x_1, \dots, x_m$ to a unique interval $[N(\mathbf{l}), N(\mathbf{u})] \subseteq [0, 1]$ that has a width of $P(x_1, x_2, \dots, x_m)$

- **Coding of $\mathbf{x} \in \{1, \dots, K\}^m$:** Start with $l = 0$ and $u = 1$. Successively iterate the following
 1. Compute $q_{\text{lower}} := P(X_j < x_j | x_1, x_2, \dots, x_{j-1})$ and $q_{\text{upper}} := P(X_j \leq x_j | x_1, x_2, \dots, x_{j-1})$
 2. Update $l \leftarrow l + q_{\text{lower}} \cdot (u - l)$ and $u \leftarrow l + q_{\text{upper}} \cdot (u - l)$

– After all data have been incorporated, compute $\mathbf{l} = N^{-1}\left(l + \frac{1}{4}(u - l)\right)$ and $\mathbf{u} = N^{-1}\left(l + \frac{3}{4}(u - l)\right)$ and return the first n bits of $\mathbf{l}$ and $\mathbf{u}$ that are the same
- **Decoding of $\mathbf{b} \in \{0, 1\}^n$:** Successively iterate the following procedure
 1. Compute $q_k := P(X_j \leq k | x_1, x_2, \dots, x_{j-1})$ for $k = 1, \dots, K$ and $q_0 = 0$
 2. Pick $x_j = k^*$ if $N(\mathbf{b}) \in [q_{k^*-1}, q_{k^*})$



Jorma Rissanen
(1932 – 2020)

**Introduction to
Probabilistic Machine
Learning**

Unit 12 – Information Theory

Arithmetic Coding: Fixed Point Arithmetic

- **Problem:** Arbitrary precision multiplication and addition in binary numbers!
- **Idea:** Fixed point arithmetic with 32-bit registers and 64-bit addition/multiplication
 1. Instead of l and u , represent l and $r := u - l$ as unsigned 32-bit integer
 2. When updating l and u , multiply them by two whenever the most significant bit of l **and** u are identical (because they cannot change anymore!)
 3. Represent the probabilities as fixed points to the basis 2^d ($d = 16$ typically)
- **Update:** If $q_{\text{lower}} \in \{0, \dots, 2^d\}$ and $q_{\text{upper}} \in \{0, \dots, 2^d\}$ then

$$l \leftarrow l + \text{UInt32} \left(\text{UInt64}(r) \cdot \frac{q_{\text{lower}}}{2^d} \right)$$

$$u \leftarrow l + \text{UInt32} \left(\text{UInt64}(r) \cdot \frac{q_{\text{upper}}}{2^d} \right)$$

- Implement all multiplications and divisions by 2 via left-/right-shifts
- **Practical Considerations:** The decoder would never stop because even a $N(\mathbf{b}) \in [q_{k^*}, q_{k^*+1})$ is true for the most likely value k !
 - Add a special symbol that indicates end-of-stream and encode it to later stop the decoder.

Arithmetic Coding: Optimal Code Length

- **Question:** How large will n be for a given data stream $\mathbf{x}$?

$$-\log_2 P(\mathbf{x}) \leq n \leq -\log_2 P(\mathbf{x}) + 1$$

- **Proof.** Assume that $\mathbf{l}$ and $\mathbf{u}$ differ first at bit $n + 1$. Because $N(\mathbf{u}) - N(\mathbf{l}) = \frac{1}{2}P(\mathbf{x})$

$$\frac{1}{2}P(\mathbf{x}) = \underbrace{\sum_{i=1}^n (u_i - l_i)2^{-i}}_{=0} + \sum_{i=n+1}^{\infty} (u_i - l_i) \cdot 2^{-i}$$

$$2^{-(n+1)} \leq 2^{-1}P(\mathbf{x}) \leq 2^{-n}$$

$$-(n+1) \leq \log_2 P(\mathbf{x}) - 1 \leq -n$$

$$n+1 \geq -\log_2 P(\mathbf{x}) + 1 \geq n$$

- **Intuitively:** Between 0 and 1 there are exactly $\frac{2}{P(\mathbf{x})}$ many intervals of length $\frac{1}{2}P(\mathbf{x})$ and one needs $\log_2 \frac{2}{P(\mathbf{x})} = -\log_2 P(\mathbf{x}) + 1$ many bits to index them with a number.

Overview

1. Basics of Information Theory
2. Probability and Compression: Arithmetic Coding
- 3. Distance Measures for Probabilities Revisited**

**Introduction to
Probabilistic Machine
Learning**

Unit 12 – Information Theory

Coding with Imperfect Information



Solomon Kullback
(1909 – 1994)



Richard Leibler
(1914 – 2003)

Introduction to Probabilistic Machine Learning

Unit 12 – Information Theory

- In practice, we often do not know the distribution P that generates our symbols but must make assumptions Q when developing codes.
- **Question:** What do we lose in compression when making this assumption?
 - Each symbol x has an “information difference” of $-\log_2(Q(x)) + \log_2(P(x))$
 - Each symbol x still appears with probability $P(x)$

$$\begin{aligned}\text{CompressionLoss}[P, Q] &= \sum_x P(x) \cdot [\log_2(P(x)) - \log_2(Q(x))] \\ &= \sum_x P(x) \cdot \log_2 \left[\frac{P(x)}{Q(x)} \right]\end{aligned}$$

- The compression loss is the *Kullback-Leibler divergence* between P and Q !
- Two properties of Kullback-Leibler Divergence:
 1. In general, $\text{KL}[P, Q] \neq \text{KL}[Q, P]$ (each summand is negated, and the weightings change!)
 2. $\text{KL}[P, Q] = 0$ if and only if $P = Q$ (each summand is zero)

(Relative) Entropy of Continuous Random Variables

- **Question:** What is entropy of a continuous random variable?
- **Differential Entropy.** *The differential entropy of a continuous random variable X is the average level of information inherent to the variable's outcomes*

$$H_b[X] := - \int p(x) \cdot \log_b(p(x)) \, dx = E_{X \sim P}[-\log_b(p(X))]$$

- Differential entropy can be negative and has no interpretation!
- **Relative Entropy.** The KL divergence between two densities *does* have an interpretation in terms of bits (base 2) or nats (base e) because

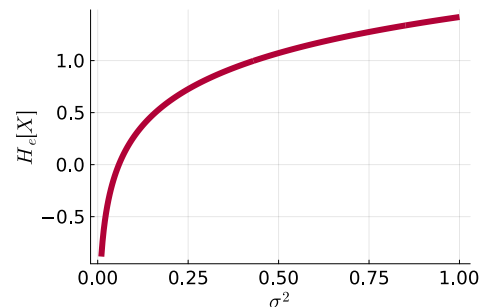
$$\text{KL}_b[p, q] = \int p(x) \cdot \left[\log_b \left(\frac{p(x)}{q(x)} \right) \right] dx \geq 0$$

- **Proof:** Using Jensen's inequality for the convex function $-\log_b(t)$ we see that

$$\int p(x) \cdot \left[\log_b \left(\frac{p(x)}{q(x)} \right) \right] dx = \int p(x) \cdot \left[-\log_b \left(\frac{q(x)}{p(x)} \right) \right] dx \geq -\log_b \left(\int p(x) \cdot \frac{q(x)}{p(x)} dx \right) = 0$$

$$X \sim \mathcal{N}(x; \mu, \sigma^2)$$

$$H_e[X] = \frac{1}{2} [\log(2\pi\sigma^2) + 1]$$



1. Information Theory

- Information theory measures the amount of uncertainty and only depends on probabilities
- The information of each outcome of a probability distribution is $-\log_2(P(x))$ bits and entropy is the expected information across all outcomes
- Entropy is the smallest size a file can be compressed to without loss (coding theory!)

2. Arithmetic Coding

- Compression algorithm that maps each sequence of tokens to its probability under a given probabilistic model
- Using fixed point arithmetic for arbitrary-precision numbers is key
- Achieves a near-optimal encoding/compression length (up to one bit!)

**Introduction to
Probabilistic Machine
Learning**

Unit 12 – Information Theory

3. Kullback-Leibler Divergence

- Measures the average loss in compression when the data comes from a distribution P but we use the distribution Q for compression (with arithmetic coding)

See you next week!